SENG 201
Software Engineering Foundations

[Week 2, Tuesday]



Yesterday...

```
/*
 * Warning:
 * In the real world,
 * NEVER use floating-point arithmetic
 * to compute prices.
 */

class CostCalculator {

    int discountThreshold = 100;
    double discount = 0.10;
    double gstRate = 0.15;

    void main() {
        showPurchaseInformation("Watermelons (for math class)", 30, 150, true);
        showPurchaseInformation("Laptop", 2500, 1, false);

        String[] penColors = { "blue", "black", "red", "green", "pink" };
        for (String penColor : penColors) {
            showPurchaseInformation(penColor + " pens", 1.5, 200, true);
        }
    }

    double computePrice(int nbItems, double unitPrice) {
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Today: Understand single-class syntax

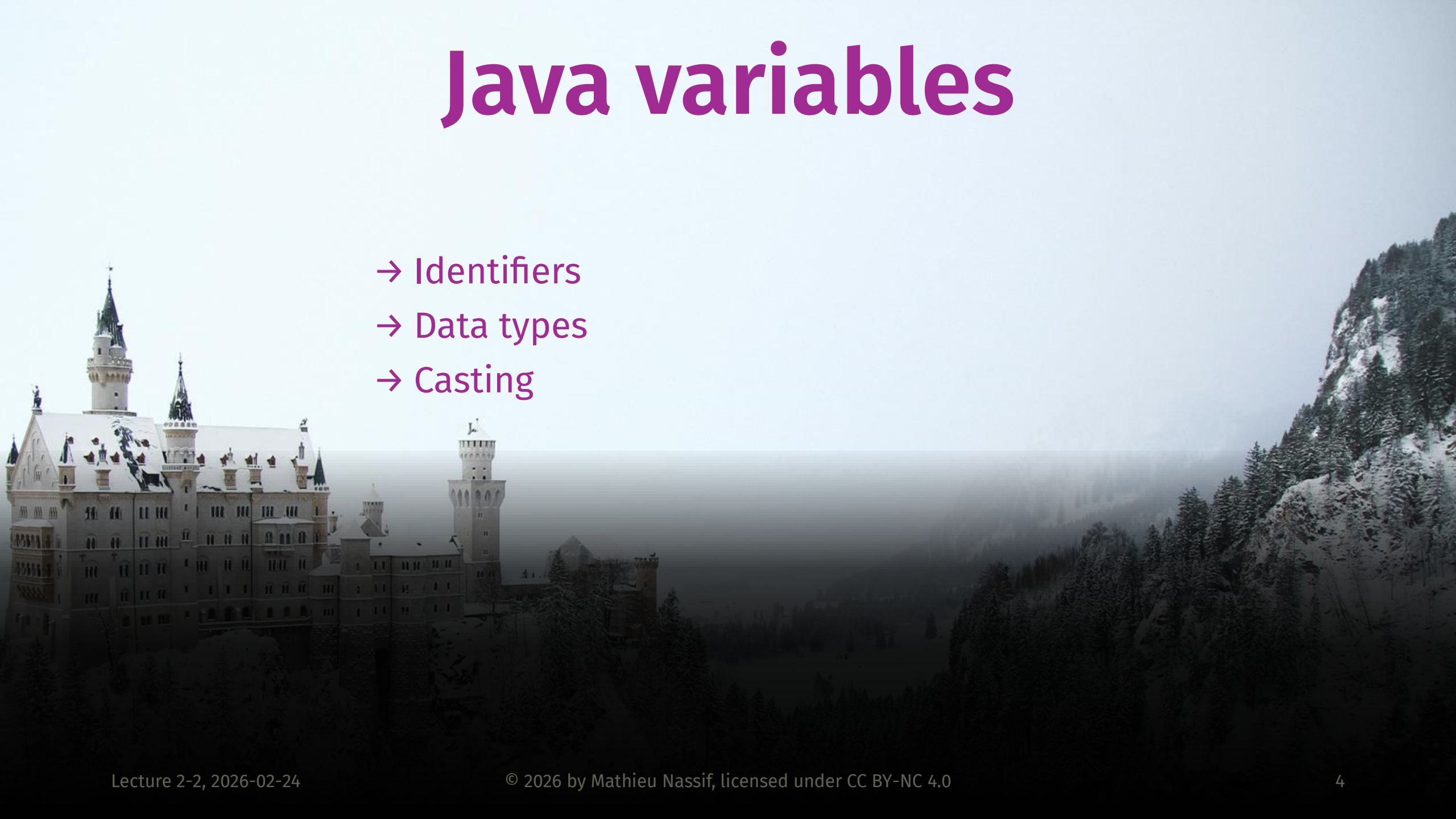
Identifiers • Data types • Declarations • Operators • Strings

Tomorrow: Multiple classes

Classes and objects • Constructors • Static • Types • Scope

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Java variables



→ Identifiers

→ Data types

→ Casting

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Identifiers: Naming things in code



A **symbol** used that identifies a program **entity**

(lexical token)

(names)
(maps to)

- Variable
- Method
- Class
- …

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Identifiers: Naming things in code

https://docs.oracle.com/javase/specs/jls/se25/jls25.pdf

Rules

- Letters, digits, underscore (_), dollar sign ($)
- Support Unicode letters (beware of encoding)
- Does NOT start with a digit
- NOT a keyword (class, int, ...), true, false, null
- Case-sensitive

ReservedKeyword:
(one of)

abstract	continue	for	new	switch
assert	default	if	package	synchronized
boolean	do	goto	private	this
break	double	implements	protected	throw
byte	else	import	public	throws
case	enum	instanceof	return	transient
catch	extends	int	short	try
char	final	interface	static	void
class	finally	long	strictfp	volatile
const	float	native	super	while

_ *(underscore)*

ContextualKeyword:
(one of)

exports	opens	requires	uses	yield
module	permits	sealed	var	
non-sealed	provides	to	when	
open	record	transitive	with	

Conventions

- camelCase / PascalCase (ALL_CAPS for constants)
- Descriptive, avoid over abbreviating
- Project-specific conventions (e.g., Hungarian notation)

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Conventions by type of identifier

Variables

- Nouns
- Start with lowercase

bike, wheelyTransport

Methods

- Verbs
- Start with lowercase

go, drawSheepForPrince

Classes

- Nouns
- Start with uppercase

Node, AiSolver

Class Constants

- Nouns
- All uppercase

CAR, ANGRY_HORSE

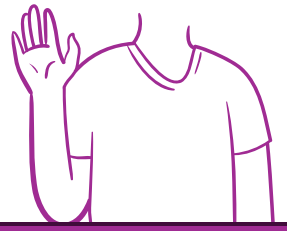
Packages

- Reversed domain name
- All lowercase

nz.ac.canterbury.seng201

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Let’s test it…



Why care about style?

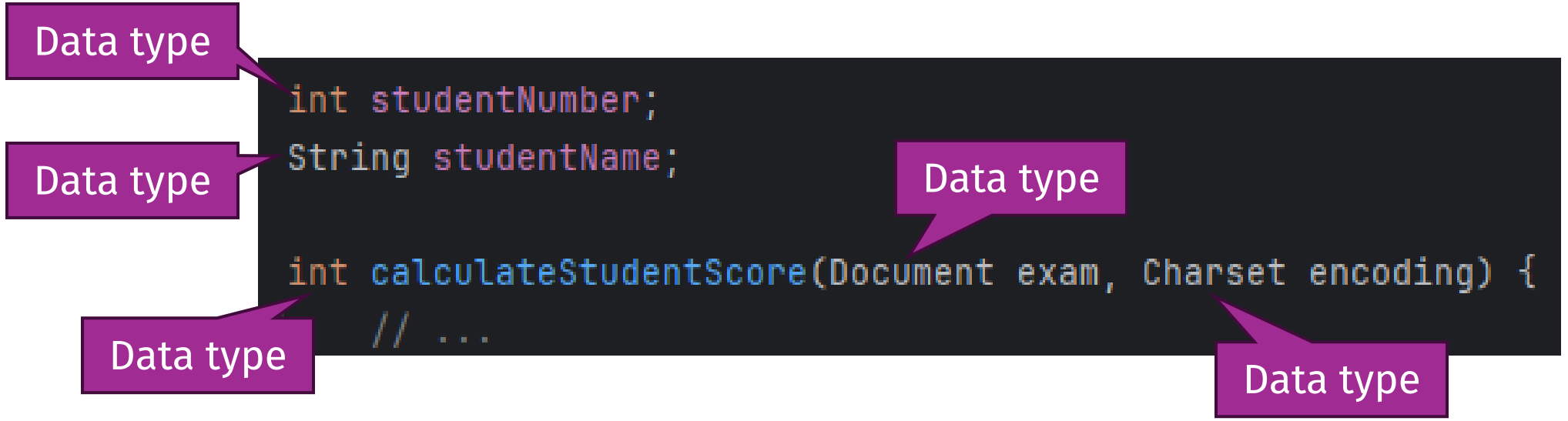
- Readability
- Understandability
- Form of documentation

Good	Poor style	Illegal
	(var) num	
		(var) length-of-text
(class) BankAccount		
	(method) MoveRight	
	(method) µmLength	
	(method) calcXYSq	

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Type checking

Java is a *strongly typed* language



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Type checking

Dynamic / weak type checking (Python, ...)

- Untyped variables
- Typed values
- Checking at run time

Static / strong type checking (Java, ...)

- Typed variables
- Typed values
- Checking at compile time

See https://instil.co/blog/static-vs-dynamic-types/ for a discussion of both styles.

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Type checking

Python

```python
def printInfo(item, price):
    print(item + ': ' + str(price) + ' NZD')


printInfo(2000, 'Laptop')
```

Java

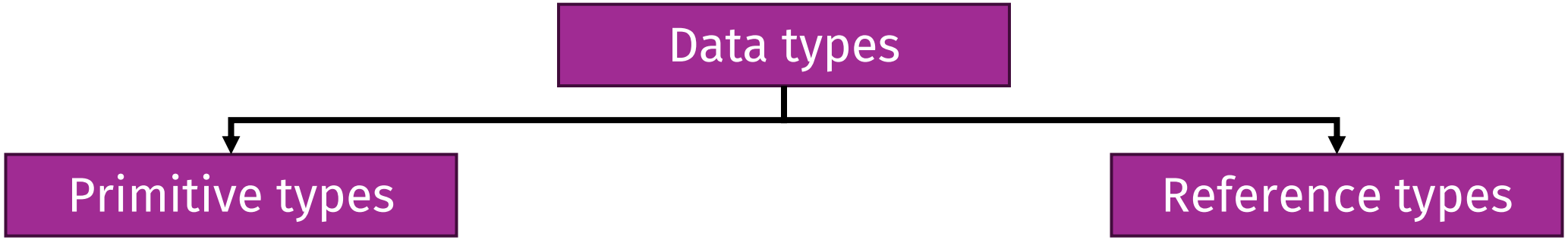
```java
void printInfo(String item, int price) {
    System.out.println(item + ": " + price + " NZD");
}

printInfo(2000, "Laptop");
```

See https://instil.co/blog/static-vs-dynamic-types/ for a discussion of both styles.

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Java data types



Integers
- `int`
- `long` (large integers)
- `byte`, `short` (constrained memory)
- `char` (characters, e.g. `'a'`)

Numbers
- `double`
- `float` (constrained memory)

Truth value
- `boolean`

Open-ended
- Each class create a new type

Standard API
- `String`
- `List`, `Set`, `Map`

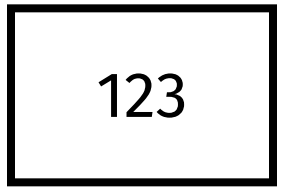
Boxed types
- `Integer`, `Double`, …

Arrays
- `int[]`, `String[]`, …

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Java data types

Value type, e.g.,
`int studentNumber`



The value `null` can only be assigned to reference types.

Reference type, e.g.,
`String studentName`



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Wrapper classes

- Use primitive types as objects
- Provide useful constants and methods

Primitive type	Boxed type
int	Integer
short	Short
byte	Byte
long	Long
char	Character
float	Float
double	Double
boolean	Boolean

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Type conversions (casting)

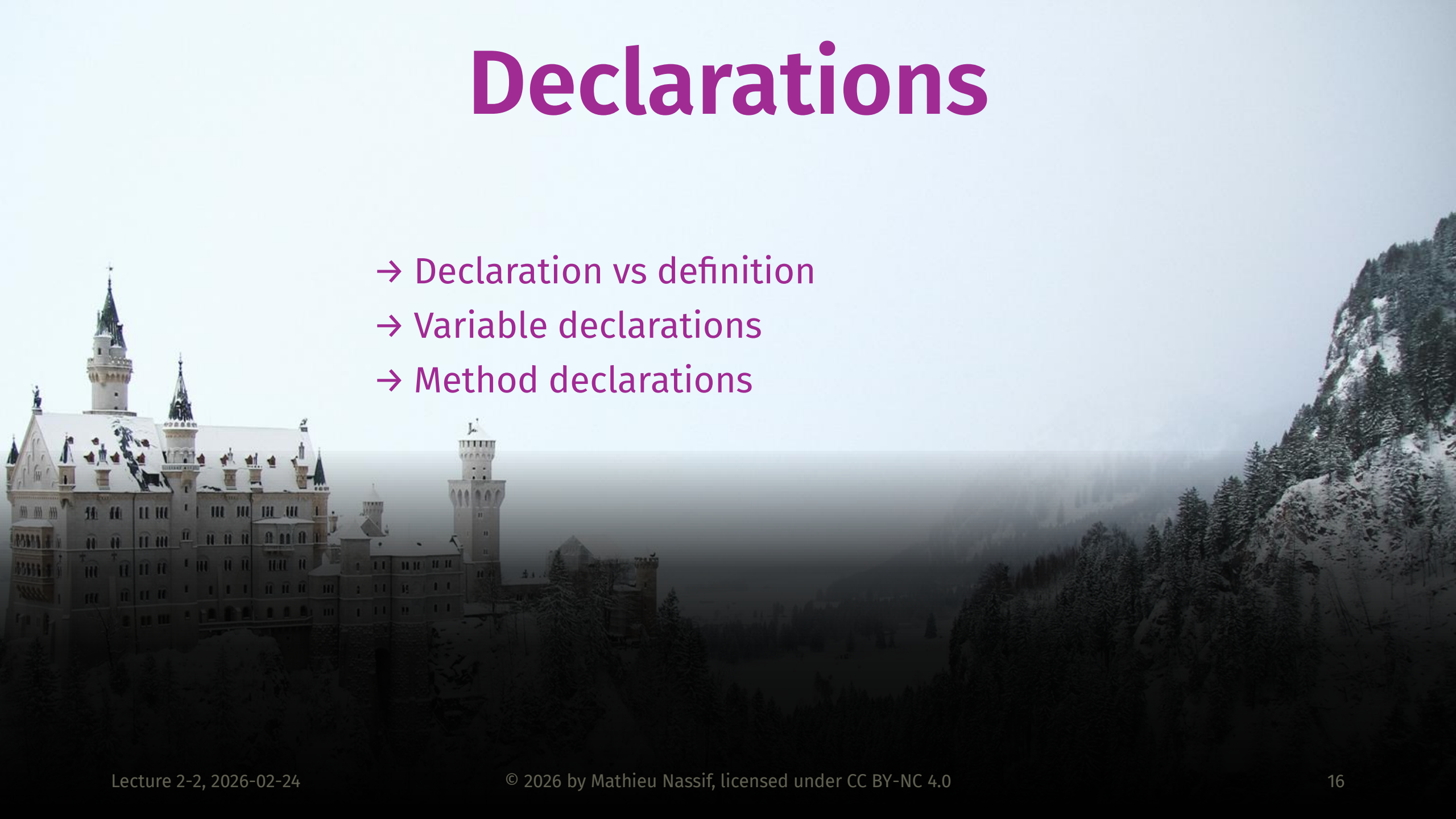
- Implicit **widening** primitive or reference conversion
- Auto-**boxing** and **unboxing**
- Explicit conversion: `(Type) expression`

* Conversion between numbers and String use wrapper class methods.

	i: int	s: short	l: long	d: Double	str: String
`int x =`	`i`	`s`	`(int) l`	`(int)(double) d`	illegal*
`double x =`	`i`	`s`	`l`	`d`	illegal*
`Integer x =`	`i`	`(int) s`	`(int) l`	illegal	illegal*
`Number x =`	`i`	`s`	`l`	`d`	illegal*

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Declarations



→ Declaration vs definition

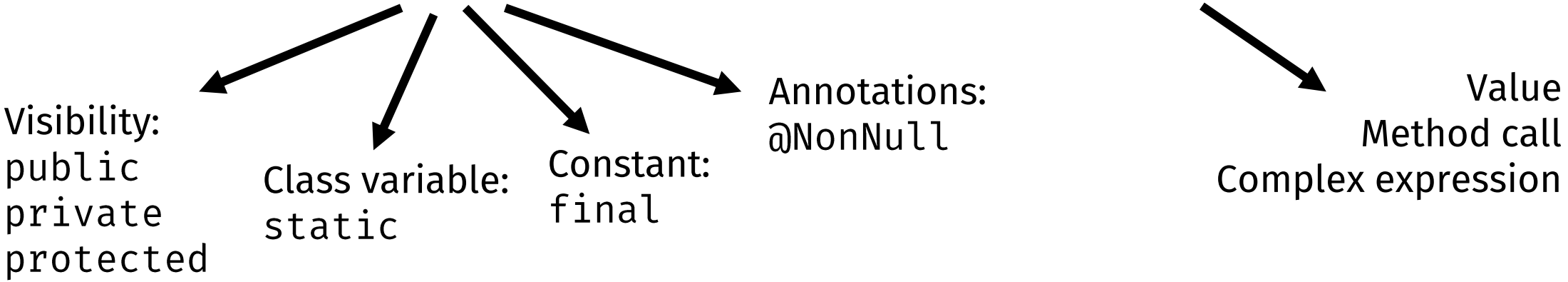
→ Variable declarations

→ Method declarations

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Syntax of variable declarations

Allowed modifiers depend on context



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Syntax of variable declarations

[modifiers*] type identifier [= expression] ;

```
public static final String MY_NAME = "Mathieu";
static Object serializedInput;
private int[] dataPoints = fetchData();
double average = DataStore.getDataset().computeAverage();
List<Integer> studentIds = new ArrayList<>();
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Local variables and class variables

```
class SomeClass {

    int classVariable = 0;
    int field = 1;
    int attribute = 2;

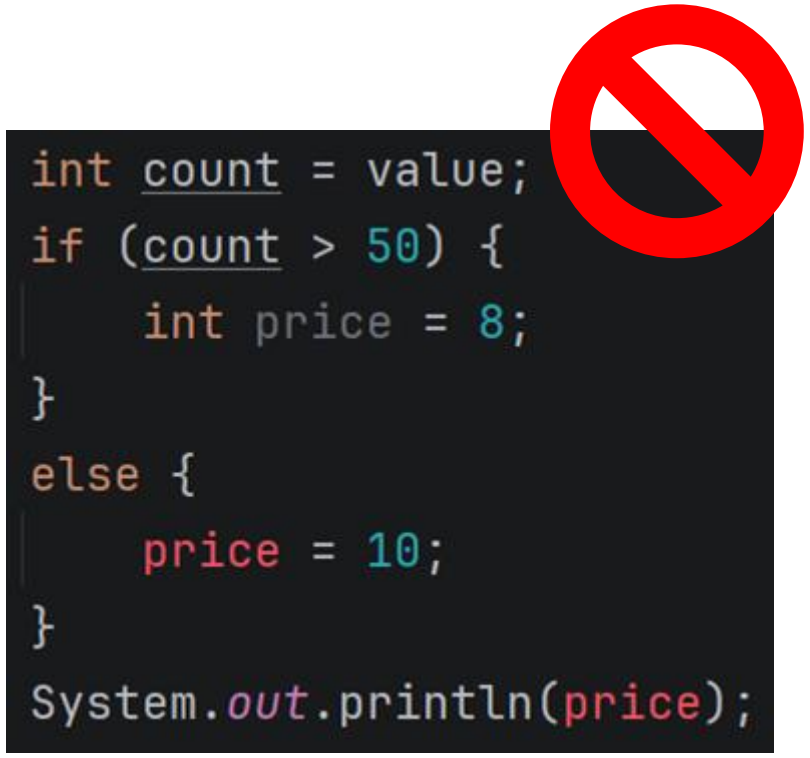
    void method(int parameter) {
        int localVariable = 3;
        // ...
```

Class variables: throughout the class

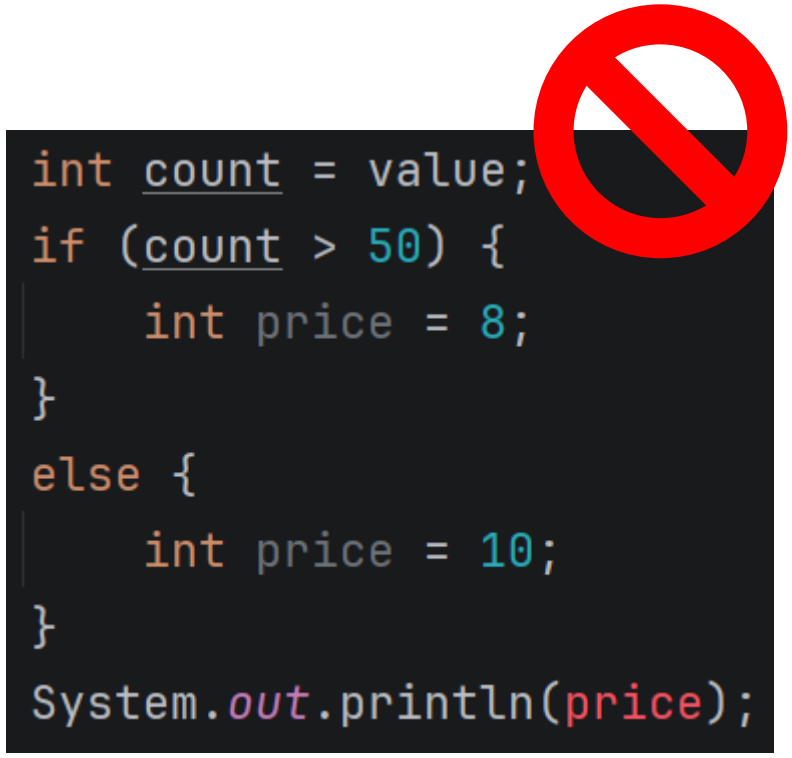
Local variables: closest block

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

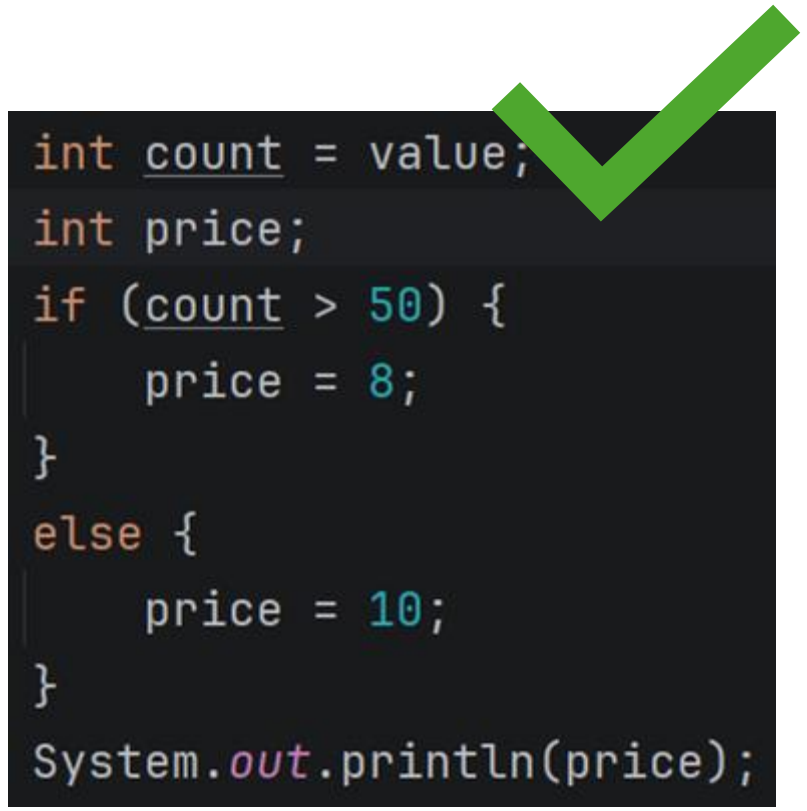
Local variables and class variables



```
int count = value;
if (count > 50) {
    int price = 8;
}
else {
    price = 10;
}
System.out.println(price);
```



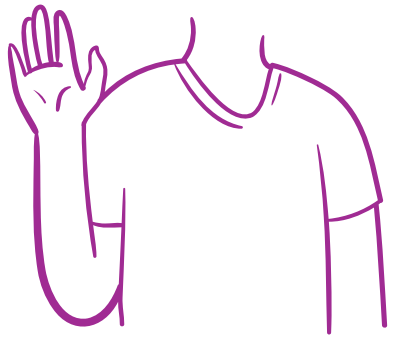
```
int count = value;
if (count > 50) {
    int price = 8;
}
else {
    int price = 10;
}
System.out.println(price);
```



```
int count = value;
int price;
if (count > 50) {
    price = 8;
}
else {
    price = 10;
}
System.out.println(price);
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

What's the difference?



`int x = 1;`	Declaration + initialization	Both
`int x;`	Declaration	Create the variable
`x = 1;`	Assignment	Change/set the value

Class variables are initialized to default values (0, false, null). Local variables must be initialized before use.

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Constants: final modifier

- Must be explicitly assigned once
- Cannot be reassigned
- Not (necessarily) immutable
- CAPITAL_CASE



immutable: object will not change

final: variable will always point to the same object

```
public static final String NAME = "Mathieu";
public static final int USER_CODE = 123;
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Syntax of method declarations

```
[modifiers*] returntype identifier ( [parameters*] ) [throws clause]
             {
                [body]
             }
```



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Parameters

[modifiers*] type identifier

No default value

[modifiers*] type... identifier

Variable arity “varargs”

- Must be the last parameter
- Equivalent to an array type

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Variable arity methods

```java
double average(Double... values) {
    double sum = 0;
    for (double value : values) {
        sum += value;
    }
    return sum / values.length;
}
```

```java
average(1.0, 2.0, 3.0, 4.0, 5.0);
Double[] values = { 1.0, 2.0, 3.0 };
average(values);
average();
average(1.0);
average(null);
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Method body and return

- Return statement to end the method and indicate the result

```
return expression ;
```

- No method output: optional return statement, use type `void`

```
return ;
```

- Does not allow (directly) nested methods

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Let’s put it all together

```
public static void main(String[] args) {
    System.out.println(Arrays.toString(args));
}

private boolean drawFigures(Canvas canvas, Shape... shapes)
        throws IllegalStateException {
    // ...
    return false;
}

protected void print(String message, boolean error) {
    if (message == null) {
        return;
    }
    if (error) System.err.println(message);
    else System.out.println(message);
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Method overloading

Overloading: same name, different parameter types

```
String print(String message, int[] addresses)
```

Overloading

```
String print(String message)

boolean print(String msg, int addr)

String print(Object msg, int[] addr)
```

Illegal

```
boolean print(String msg, int[] addr)

String print(String msg, int... addr)
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

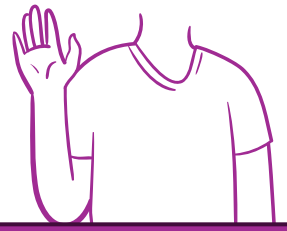
Method overloading

Use with care!

```
void overload(String s, int... i) { System.out.println("int..."); }
void overload(String s, Integer... i) { System.out.println("Integer..."); }
void overload(String s, Object[] o) { System.out.println("Object[]"); }
void overload(String s, Object o) { System.out.println("Object"); }
void overload(String s) { System.out.println("String"); }
```

```
Integer[] ints = { 1, 2, 3 };
Object[] objects = ints;
overload("", 1);
overload("", 1, 2, 3);
overload("", ints);
overload("", objects);
overload("", null);
overload("");
```

* Object is the ancestor class of all Java objects



What does this program print?

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Syntax of class declarations

```
[modifiers*] class identifier [extends Parent] [implements Interfaces*]
{
    [members*]
}
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Operators and Strings



→ Arithmetic operators

→ Boolean operators

→ Strings and string operations

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Java (common) operators

Arithmetic

Operator	Meaning
`x++  x--`	increment/decrement, return original value
`++x  --x`	increment/decrement, return new value
`*  /  %`	multiply, divide, modulo
`+  -`	add, substract

Assignment

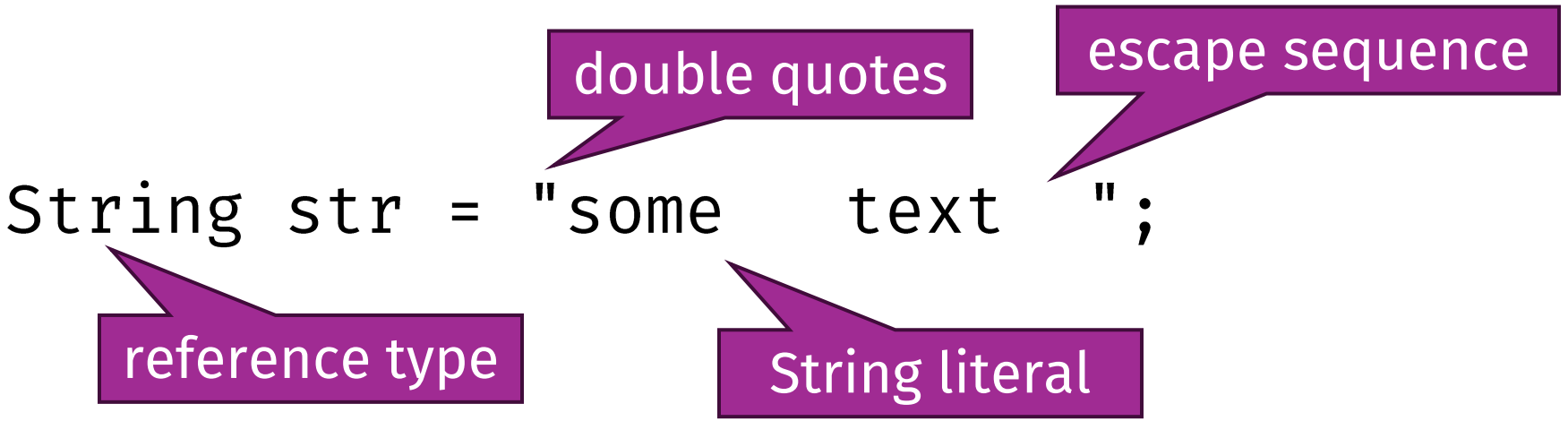
Operator	Meaning
`x = y`	assign y to x
`+=  -=  *=  /=  %=  &=  |=` (`x op= y`)	compute x op y, then assign to x

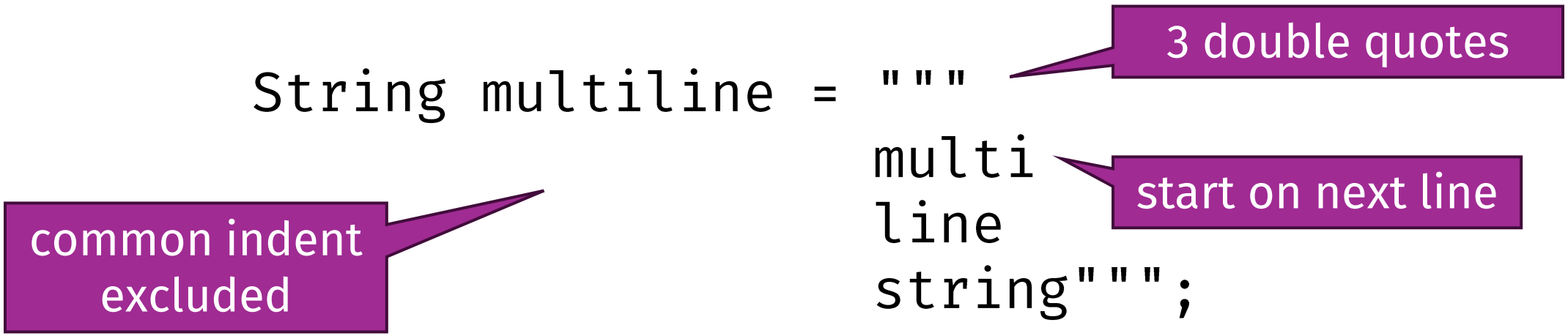
Logical

Operator	Meaning
`!`	NOT
`<  >  <=  >=`	numeric comparison
`&&`	AND (short-circuiting)
`||`	OR (short-circuiting)
`==  !=`	identity equal/unequal

© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

Strings





© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

String operations

Strings are immutable. All operations return a new String object.

- Concatenation

  ```
  "seng" + "201"
  ```

- Split

  by index: `"my string".substring(start, end)` (end is optional)

  by delimiter: `"path/to/file".split("/")` (regex)

  single character: `"abc".charAt(0)`

- Change case

  ```
  "lowercase".toUpperCase()
  "UPPER".toLowerCase()
  ```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

String operations

- Object to String

  Arrays: `Arrays.toString(someArray)`

  Objects: `myObject.toString()` `// or "" + myObject`

- Primitive types to String

```
Integer.toString(12)
Boolean.toString(false)
```

- String to primitive type

```
Integer.parseInt("12")      // return int
Integer.valueOf("12")       // return Integer
```


© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0

RECAP

→ Data types

→ Declarations

→ Strings



© 2026 by Mathieu Nassif, licensed under CC BY-NC 4.0